

Reviewer Pack — Noncommutative L^p Geometry Tensor Invariant Bounds Communic...

Entry b6c4880f850d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 05:53:25 UTC

Noncommutative L^p Geometry Tensor Invariant Bounds Communication Complexity of Disjointness

Entry ID: b6c4880f850d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 05:53:25 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (L^p Geometry) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any $n \geq 2$, there exists a constant $C(n)$ such that the noncommutative L^p spectral norm of the tensor product of two random $n \times n$ matrices in $M_2(\mathbb{C})$, with $p \in [1, \infty]$, is lower bounded by $C(n)$ times the randomized communication complexity of the disjointness function.’, ‘Equivalently, $\tau_{L^p(M)}^2 \geq C(n) * CC_R(DISJ_n)$, where $\tau_{L^p(M)} := ||M||_{L^p}$ for a matrix M and CC_R is the randomized two-party communication complexity.’]

Rationale (proposer’s reasoning):

[‘The noncommutative L^p geometry provides a framework to study quantum information theory, which has implications for classical communication complexity.’, ‘Tensor products of matrices can encode complex interactions between parties in communication tasks, and their spectral properties may capture the difficulty of computing certain functions like disjointness.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4083e08d0bdb24b7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The noncommutative L^p spectral norm of the tensor product of two matrices is lower bounded by $C(n)$ times the randomized communication complexity if the average spectral norm across 30 seeds exceeds $C(n)$ times the average communication complexity.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative geometry L^p " AND communication complexity AND disjointness" - "tensor product spectral norm L^p " AND randomized communication complexity AND disjointness" - "disjointness function" AND "communication complexity" AND noncommutative L^p geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiply(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def spectral_norm(matrix, p=2):
    n = len(matrix)
    if p == 1:
        max_col_sum = max(sum(abs(matrix[j][i]) for j in range(n)) for i in range(n))
        max_row_sum = max(sum(abs(matrix[i][j]) for i in range(n)) for j in range(n))
        return max(max_col_sum, max_row_sum)
```

```

elif p == 2:
    A = [[matrix[i][j] * matrix[i][j] for j in range(n)] for i in range(n)]
    eigenvalues = []
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    for _ in range(100):
        x = [random.gauss(0, 1) for _ in range(n)]
        x_norm = math.sqrt(sum(x[i]**2 for i in range(n)))
        x = [x[i] / x_norm for i in range(n)]
        y = matrix_multiply(matrix, x)
        y_norm = math.sqrt(sum(y[i]**2 for i in range(n)))
        eigenvalue = sum(x[i] * y[i] for i in range(n)) / (y_norm * x_norm)
        eigenvalues.append(eigenvalue)
    return max(eigenvalues)**0.5
else:
    raise ValueError("Unsupported p value")

def communication_complexity_disjointness(n):
    # Simplified model of communication complexity for disjointness function
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10 # Start with a small size and increase if needed
    C_n = 1.0 / n # Example constant, adjust as needed
    instances_tested = 30

    total_spectral_norm = 0
    total_communication_complexity = 0

    for _ in range(instances_tested):
        M1 = [[random.gauss(0, 1) for _ in range(n)] for _ in range(n)]
        M2 = [[random.gauss(0, 1) for _ in range(n)] for _ in range(n)]

        T = matrix_multiply(M1, M2)
        spectral_norm_T = spectral_norm(T)

        communication_complexity = communication_complexity_disjointness(n)

        total_spectral_norm += spectral_norm_T
        total_communication_complexity += communication_complexity

    average_spectral_norm = total_spectral_norm / instances_tested
    average_communication_complexity = total_communication_complexity / instances_tested

    conjecture_holds = average_spectral_norm >= C_n * average_communication_complexity
    counterexample = "" if conjecture_holds else "disjointness_function"

    return {
        "metric_name": "Spectral Norm vs Communication Complexity",
        "metric_value": average_spectral_norm,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_507e0670.py", line 105, in <module>
    result = run_trial(seed)
             ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_507e0670.py", line 78, in run_trial
    spectral_norm_T = spectral_norm(T)
                      ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_507e0670.py", line 52, in spectral_norm
    y = matrix_multiply(matrix, x)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_507e0670.py", line 35, in matrix_multiply
    C[i][j] += A[i][k] * B[k][j]
               ~~~~~^
TypeError: 'float' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be verified. | next: Debug the test code to ensure it runs without errors and then re-run the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12043
2	propose	ollama_remote	glm4:latest	0	0	9408
3	propose	ollama_remote	glm4:latest	0	0	10196
4	preregistration	ollama_remote	glm4:latest	0	0	5394
5	novelty	ollama_remote	glm4:latest	0	0	4801
6	novelty	ollama_remote	glm4:latest	0	0	10420
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37027
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10315
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8071
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13516
11	judge	ollama_remote	glm4:latest	0	0	13113

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134305 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/b6c4880f850d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b6c4880f850d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b6c4880f850d.tar.gz (if generated)